

Theresia Mulyadi 陳友平 (112006221)

How the Code Works

The program implements cooperative multithreading for the 8051 microcontroller. ``Bootstrap()`` initializes the thread-management variables and creates ``main()`` as thread 0. ``ThreadCreate()`` finds an unused thread ID, prepares a separate stack for the new thread, saves the function address on the stack, initializes the register context, and updates the thread bitmap. Each thread uses a different register bank and a separate 16-byte stack area.

``ThreadYield()`` performs a context switch. It first saves the current thread's registers and stack pointer, then selects the next valid thread using round-robin scheduling. After that, it restores the selected thread's stack pointer and register values so execution can continue from where that thread previously yielded.

In ``testcoop.c``, the Producer generates characters from ``A`` to ``Z`` and stores them in ``sharedBuffer``. The ``bufferFull`` flag prevents the Producer from overwriting data before the Consumer reads it. The Consumer waits until the buffer contains data, copies the character into ``SBUF``, clears ``bufferFull``, and waits for the UART transmission to finish. Both threads call ``ThreadYield()`` while waiting, allowing the other thread to run.

3.1: make clean and make compilation screenshot

```
PS C:\Users\Theresia\Downloads\ppc1\ppc1> make clean
powershell -NoProfile -Command "Remove-Item -ErrorAction SilentlyContinue *.
hex,*.ihx,*.lnk,*.lst,*.map,*.mem,*.rel,*.rst,*.sym,*.asm,*.lk"
PS C:\Users\Theresia\Downloads\ppc1\ppc1> make
sdcc -c testcoop.c
sdcc -c cooperative.c
sdcc -o testcoop.hex testcoop.rel cooperative.rel
PS C:\Users\Theresia\Downloads\ppc1\ppc1> Get-ChildItem testcoop.*

Directory: C:\Users\Theresia\Downloads\ppc1\ppc1

Mode                LastWriteTime         Length Name
----                -
-a-----        6/13/2026   4:42 AM           13193 testcoop.asm
-a-----        6/13/2026   3:38 AM            4425 testcoop.c
-a-----        6/13/2026   4:42 AM            1198 testcoop.hex
-a-----        6/13/2026   4:42 AM             256 testcoop.lk
-a-----        6/13/2026   4:42 AM          33941 testcoop.lst
-a-----        6/13/2026   4:42 AM          16980 testcoop.map
-a-----        6/13/2026   4:42 AM            1192 testcoop.mem
-a-----        6/13/2026   4:42 AM            5467 testcoop.rel
-a-----        6/13/2026   4:42 AM          34473 testcoop.rst
-a-----        6/13/2026   4:42 AM          40162 testcoop.sym

PS C:\Users\Theresia\Downloads\ppc1\ppc1>
```

3.2

Part 1: Before ThreadCreate(main)

The screenshot displays the Proteus 8.09 SPICE simulator interface. The top section shows the assembly code window with the following instructions:

```

0096| JB 99H,05H
0099| LCALL 0140H
009C| SJMP 0F8H
009E| CLR 99H
00A0| SJMP 0E5H
00A2| MOV 31H,#00H
00A5| MOV 30H,#00H
00A8| MOV 32H,#41H
00AB| MOV DPTR, #004FH
00AE| LCALL 00DFH
00B1| LJMP 0075H
00B4| LJMP 00BBH
00B7| RET
00B8| RET
00B9| RET
00BA| RET
00BB| MOV 25H,#00H
00BE| MOV 24H,#00H
00C1| MOV 81H,#3FH
00C4| MOV DPTR, #00A2H
00C7* LCALL 00DFH

```

The register window on the left shows the following values:

R/O	W/O	TH0	TL0	R7	B
0x00	0x00	0x00	0x00	0x10	0x00
				R6	ACC
				0x00	0x00
				R5	PSW
				0x00	0x00
				R4	IP
				0x00	0x00
				R3	IE
				0x00	0x00
				R2	PCON
				0x00	0x00
				R1	DPH
				0x21	0x00
				R0	DPL
				0x20	0xA2
					SP
					0x3F

The data memory window shows the following values:

addr	0x00	0x01	0x02	0x03	0x04	0x05	0x06	0x07	0x08	0x09	0x0A	0x0B	0x0C	0x0D	0x0E	0x0F
00	20	21	00	00	00	00	00	10	21	20	00	00	00	00	00	00
10	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
20	46	56	00	00	00	00	01	41	4F	00	04	00	00	00	00	00
30	43	01	44	00	00	00	00	00	00	00	00	00	00	00	00	00
40	9C	00	01	00	01	00	01	00	00	00	00	00	00	00	00	00
50	59	00	01	00	00	00	09	00	00	00	00	00	00	00	00	00
60	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
70	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00

The hardware components window at the bottom shows the following components:

- DI, LD, AND Gate Disabled, Key Bounce Disabled, Standard, i
- 0.0 V output, Scope, DAC
- BF, 0, AC, 0x00, IR, 0x00, DR, 0x00, i
- U, No Parity, 8-bit UART @ 4800 Baud, Rx, Rx Reset, Tx, Tx Send, i
- 0.0 V Input, 1111111, ADC, MAX, MIN, Motor Enabled

The breakpoint is set at address 0x00C7, immediately before the first call to ThreadCreate, which creates the thread for main(). At this point, the stack pointer is 0x3F, the current thread ID at address 0x24 is 0x00, and the thread bitmap at address 0x25 is 0x00, showing that no threads have been created yet. During ThreadCreate(main), thread 0 is allocated, the bitmap becomes 0x01, and the address of main() together with the initial register context is pushed onto the stack starting at address 0x40. The final stack pointer is stored in savedSP[0].

Part 2: Before ThreadCreate(Producer)

The screenshot displays the Proteus IDE interface for a microcontroller simulation. The top section shows the assembly code window with the following instructions:

```

007B: MOV 89H, A
007D: MOV 8DH, #0FAH
0080: MOV 98H, #50H
0083: CLR 99H
0085: SETB 8EH
0087: MOV A, 31H
0089: JNZ 05H
008B: LCALL 0140H
008E: SJMP 0F7H
0090: MOV 99H, 30H
0093: MOV 31H, #00H
0096: JB 99H, 05H
0099: LCALL 0140H
009C: SJMP 0F8H
009E: CLR 99H
00A0: SJMP 0E5H
00A2: MOV 31H, #00H
00A5: MOV 30H, #00H
00A8: MOV 32H, #41H
00AB: MOV DPTR, #004FH
00AE: LCALL 00DFH
  
```

The register window shows the following values:

R7	0x00	B	0x00
R6	0x00	ACC	0x20
R5	0x00	PSW	0x01
R4	0x00	IP	0x00
R3	0x00	IE	0x00
R2	0x00	PCON	0x00
R1	0x21	DPH	0x00
R0	0x20	DPL	0x4F
SP	0x3F		

The PC register is highlighted at 0x00AE. The data memory window shows the following values:

0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
00	20	21	00	00	00	00	00	00	21	20	00	00	00	00	00
10	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
20	46	56	00	00	00	01	00	41	A2	00	01	00	00	00	00
30	00	00	41	00	00	00	00	00	00	00	00	00	00	00	00
40	A2	00	00	00	00	00	00	00	00	00	00	00	00	00	00
50	73	00	42	00	00	00	08	00	00	00	00	00	00	00	00
60	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
70	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00

The hardware components window shows the following components:

- DI, LD, AND Gate Disabled, Key Bounce Disabled, Standard, 8-bit UART @ 4800 Baud, No Parity, Rx Reset, Tx Send, 0.0 V input, 11111111, ADC, Motor Enabled.

The breakpoint is set at address 0x00AE, immediately before main() calls ThreadCreate(Producer). At this point, currentThread at address 0x24 is 0x00, indicating that thread 0 is running, and the thread bitmap at address 0x25 is 0x01, showing that only thread 0 is currently active. DPTR contains 0x004F, which is the address of the Producer() function. During ThreadCreate(Producer), thread ID 1 is allocated and the bitmap changes to 0x03. The stack pointer is temporarily moved to 0x4F, so the new Producer stack begins at address 0x50. The function address and initial values for ACC, B, DPL, DPH, and PSW are pushed onto the stack. PSW is initialized to 0x08 to select register bank 1, and the completed stack pointer is saved in savedSP[1].

Part 3: Producer Running

The screenshot shows the Proteus IDE interface. The top window displays the assembly code and the current instruction being executed. The bottom window shows the hardware simulation, including a keypad, a display, and various control buttons.

Assembly Code:

```
0000E: MOV 31H, #01H | Time: 255us
00034: JB 0F5H, 0D3H
00037: MOV A, R2
00038: XCH A, 0R0
00039: MOV 82H, A
0003B: RET
0003C: MOV 82H, R3
0003E: RET
0003F: JNB 0F6H, 0E6H
00042: MOV R0, 82H
00044: JB 0F5H, 0D9H
00047: SJMP 0CFH
00049: LJMP 0A2H
0004C: LJMP 0A9H
0004F: MOV 32H, #41H
00052: MOV A, 31H
00054: JZ 05H
00056: LCALL 0140H
00059: SJMP 0F7H
0005B: MOV 30H, 32H
0005E: MOV 31H, #01H
00061: MOV A, #5AH
```

Hardware Simulation:

- Keypad:** A 4x4 grid of buttons labeled 0-9, *, #, and letters A-D.
- Display:** A 4-digit 7-segment display showing '8888'.
- Control Buttons:** 'AND Gate Disabled', 'Key Bounce Disabled', 'Standard', 'Rx Reset', 'Tx Send', 'ADC', 'Motor Enabled'.
- ADC:** A vertical slider labeled '0.0 V input' and '11111111'.
- Motor:** A circular motor icon labeled 'MAX' and 'MIN'.

The Producer is running because the program counter is within the `_Producer` function, and `currentThread` at address `0x24` contains `0x01`, indicating thread 1. The `PSW` value is `0x08`, which selects register bank 1. The shared buffer at address `0x30` contains `0x41`, representing the character 'A', and `bufferFull` at address `0x31` is `0x01`, confirming that the Producer has written a new character into the buffer.

Part 4: Consumer Running

The screenshot shows the Proteus IDE interface with the following components:

- Top Window (Assembly Code):** Displays the assembly code for the program. The current instruction is `MOV 99H,30H` at address `0x0090`. The program counter (PC) is `0x0093`. The register dump shows `R0 = 0x20`, `R1 = 0x21`, `R2 = 0x00`, `R3 = 0x00`, `R4 = 0x00`, `R5 = 0x00`, `R6 = 0x00`, `R7 = 0x10`. The memory dump shows `0x00 = 0x41`, `0x01 = 0x00`, `0x02 = 0x00`, `0x03 = 0x00`, `0x04 = 0x00`, `0x05 = 0x00`, `0x06 = 0x00`, `0x07 = 0x00`, `0x08 = 0x00`, `0x09 = 0x00`, `0x0A = 0x00`, `0x0B = 0x00`, `0x0C = 0x00`, `0x0D = 0x00`, `0x0E = 0x00`, `0x0F = 0x00`.
- Bottom Window (Hardware Simulation):** Shows the hardware components and their status:
 - Keypad:** A 4x4 keypad with buttons labeled 1-9, 0, *, #, and function keys.
 - 7-Segment Display:** A 7-segment display showing the hexadecimal value `0x00`.
 - Control Buttons:** Buttons for `DI`, `LD`, `Scope`, `DAC`, `ADC`, `Motor Enabled`, `Rx Reset`, and `Tx Send`.
 - Registers:** A row of registers labeled `BF`, `AC`, `IR`, `DR`, and `PC` with their current values.

The Consumer is running because the program counter is within the `_Consumer` function and `currentThread`, stored at address `0x24`, is `0x00`, indicating thread `_0`. The character in `sharedBuffer` at address `0x30` is `0x41`, which represents 'A', and the same value has been copied into `SBUF`. The value of `bufferFull` at address `0x31` is `0x00`, showing that the Consumer has consumed the character and released the shared buffer for the Producer.